Case Study

Cache Memory Performance Analysis

Student Name	RITIKA GARG
UID	25BAI10013
Course Name	Computer Organization and Architecture
Faculty Name	DR. RUCHIKA GUPTA
Department	B.E. CSE (AI)
Institute	University Institute of Engineering (UIE)
University	Chandigarh University
Academic Year	2026–27

1. Introduction

The CPU of a modern computer works at a speed that main memory (RAM) cannot match. If the processor had to wait for RAM on every instruction fetch and every data read, a large part of its time would be wasted in waiting. Computer designers solve this problem by placing a small, very fast memory called the cache between the CPU and the RAM.

The cache keeps copies of the instructions and data that were used recently. When the CPU asks for something that is already in the cache, the answer comes back very quickly. When the item is not there, it must be brought from the slower RAM, and this costs additional time. The overall speed of the memory system therefore depends on how often the CPU finds what it wants in the cache.

This case study uses the following values to measure how good the cache is: total memory accesses = 2,000, cache hits = 1,640, cache misses = 360, cache hit time = 3 ns and additional miss penalty = 60 ns.

2. Objectives

This case study is meant to:

- First, explain what cache memory is and where it is located between the CPU and RAM.
- Then, clarify the difference between a cache hit and a cache miss.
- Next, figure out the hit ratio and miss ratio using the provided information.
- After that, calculate the Average Memory Access Time (AMAT).
- Then, determine how much the cache helps improve performance.
- Also, look into how the hit ratio influences the average access time.
- Finally, learn about the concept of locality of reference.

3. Problem Statement

A processor makes 2,000 memory accesses. Of these, 1,640 are found in the cache (hits) and the other 360 are not (misses). A hit takes 3 ns, and every miss adds a further 60 ns to fetch the data from RAM. Using this data, the effectiveness of the cache has to be calculated, and the link between the hit ratio and the average memory access time has to be explained.

4. Case Study Questions

Q1. What is cache memory?

Cache memory is a small and extremely fast type of memory that is located between the CPU and the main memory. It stores copies of the instructions and data that the processor has recently used or is likely to need soon, allowing them to be accessed much faster than from regular RAM.

A cache is needed because the CPU is much quicker than RAM.
The cache acts as a fast buffer that helps reduce the big difference in speed between the CPU and RAM. Its main features are:

Speed: It is made using SRAM and placed very close to the CPU, making it much faster than RAM.

Size: It is small, ranging from a few kilobytes to a few megabytes, because fast memory is expensive.

Automatic operation: The hardware automatically decides what data to keep in the cache; the programmer doesn't have control over this.

Levels: Caches come in different levels—L1 (smallest and fastest), L2, and L3 (larger and slower).

Benefit: It reduces the amount of time the CPU has to wait for data from memory.

In this case study, the cache provides data in 3 nanoseconds, while a cache miss adds 60 nanoseconds to retrieve the data from RAM.

Q2. Differentiate between a cache hit and a cache miss.

Cache hit: The CPU asks for some data and finds it already stored in the cache. The data goes straight to the CPU in the hit time, which is 3 ns here.

Cache miss: The requested data is not in the cache. The CPU waits while the data is read from RAM and a copy is placed in the cache. A miss therefore takes the hit time plus the miss penalty, i.e. 3 ns + 60 ns = 63 ns.

The comparison is given in the table below.

Basis	Cache Hit	Cache Miss
Meaning	Data is present in the cache	Data is absent from the cache
Source of data	Cache	RAM (then copied to cache)
Time taken	3 ns	3 + 60 = 63 ns
Count in this case	1,640 of 2,000	360 of 2,000
Effect on speed	Fast, improves performance	Slow, lowers performance
Designer's aim	Make it as frequent as possible	Keep it as rare as possible

Q3. Calculate the cache hit ratio.

The hit ratio tells what fraction of all memory accesses are satisfied by the cache.

> **Hit Ratio = Number of Cache Hits ÷ Total Memory Accesses**

Step 1: Given values

- Cache hits = 1,640
- Total memory accesses = 2,000

Step 2: Substitute

- Hit Ratio = 1,640 ÷ 2,000

Step 3: Calculate

- Hit Ratio = 0.82
- In percentage: 0.82 × 100 = 82%

Answer: The hit ratio is 0.82 (82%). So the CPU finds its data in the cache for 82 out of every 100 accesses.

Q4. Calculate the cache miss ratio.

The miss ratio is the fraction of memory accesses for which the data is not in the cache.

> **Miss Ratio = Number of Cache Misses ÷ Total Memory Accesses**

Step 1: Given values

- Cache misses = 360
- Total memory accesses = 2,000

Step 2: Substitute and calculate

- Miss Ratio = 360 ÷ 2,000 = 0.18
- In percentage: 0.18 × 100 = 18%

Step 3: Cross-check

- Miss Ratio = 1 − Hit Ratio = 1 − 0.82 = 0.18
- Hit Ratio + Miss Ratio = 0.82 + 0.18 = 1, so the values are consistent.

Answer: The miss ratio is 0.18 (18%).

Q5. Why is locality of reference important?

Locality of reference means that programs tend to use the same memory locations, or locations close to them, repeatedly over a short period. This behaviour is the reason a small cache can be so useful. It appears in two forms:

- **Temporal locality:** an item used just now is likely to be used again soon, for example a loop variable that is read in every iteration.
- **Spatial locality:** if one location is used, its neighbours are likely to be used next, for example the elements of an array that is processed in order.

It is important for the following reasons:

- The cache can store only a tiny part of RAM, so it must choose carefully; locality says that recently used data is the best choice.

- On a miss, a whole block of nearby data is brought in, so the following accesses are likely to be hits.
- Good locality gives a high hit ratio, and a high hit ratio gives a low AMAT.
- If programs had no locality, most accesses would miss and the cache would be of little use.

The 82% hit ratio in this case study is possible only because the processor's accesses show good temporal and spatial locality.

5. Assignment

5.1 Calculation of AMAT

Given

- Total accesses = 2,000; hits = 1,640; misses = 360
- Hit time = 3 ns; additional miss penalty = 60 ns

The Average Memory Access Time is found from:

> **AMAT = Hit Time + (Miss Rate × Miss Penalty)**

Step 1: Miss rate

- Miss Rate = 360 ÷ 2,000 = 0.18

Step 2: Contribution of misses

- Miss Rate × Miss Penalty = 0.18 × 60 = 10.8 ns

Step 3: Add the hit time

- AMAT = 3 + 10.8 = 13.8 ns

Step 4: Verification using total time

- Time for 1,640 hits = 1,640 × 3 = 4,920 ns
- Time for 360 misses = 360 × (3 + 60) = 360 × 63 = 22,680 ns
- Total time = 4,920 + 22,680 = 27,600 ns
- Average per access = 27,600 ÷ 2,000 = 13.8 ns

Both methods give 13.8 ns, so the calculation is correct.

5.2 Cache effectiveness

Effectiveness shows how much the cache improves memory performance. To measure it, the AMAT is compared with the time needed without a cache. It is assumed that without a cache every access goes to RAM and takes 60 ns.

Step 1: Hit ratio

- Hit Ratio = 1,640 ÷ 2,000 = 0.82 = 82%

Step 2: Speedup

- Speedup = Time without cache ÷ AMAT with cache = 60 ÷ 13.8 = 4.35 (approximately)

Step 3: Reduction in access time

- Reduction = (60 − 13.8) ÷ 60 × 100 = 46.2 ÷ 60 × 100 = 77%

Conclusion: The cache makes memory access about 4.35 times faster and lowers the average access time by 77%. If the full miss time of 63 ns is used for comparison instead, the speedup is 63 ÷ 13.8 = 4.57, which is close to the first value.

5.3 Summary of results

Quantity	Value
Hit ratio	0.82 (82%)
Miss ratio	0.18 (18%)
Miss penalty contribution (0.18 × 60)	10.8 ns
Average Memory Access Time (AMAT)	13.8 ns
Speedup compared with no cache	4.35 times
Reduction in access time	77%

Final Output: AMAT = 13.8 ns

6. Relationship Between Hit Ratio and AMAT

The hit ratio and the AMAT move in opposite directions. A higher hit ratio gives a lower AMAT, and a lower hit ratio gives a higher AMAT.

The reason is straightforward. A hit costs only 3 ns whereas a miss costs 63 ns. When more accesses are hits, fewer of them pay the extra 60 ns, so the average falls. Using Miss Rate = 1 − Hit Ratio, the formula can be rewritten as:

AMAT = 3 + (1 − Hit Ratio) × 60 = 63 − 60 × Hit Ratio

Check with the given data: 63 − 60 × 0.82 = 63 − 49.2 = 13.8 ns, which agrees with the earlier result. The table below shows the AMAT for other hit ratios with the same hit time and miss penalty.

Hit Ratio	Miss Rate	Miss Rate × 60 ns	AMAT
70%	0.30	18.0 ns	21.0 ns
80%	0.20	12.0 ns	15.0 ns
82% (given)	**0.18**	**10.8 ns**	**13.8 ns**
90%	0.10	6.0 ns	9.0 ns
95%	0.05	3.0 ns	6.0 ns
99%	0.01	0.6 ns	3.6 ns
100%	0.00	0.0 ns	3.0 ns

The table shows that each 1% rise in hit ratio lowers the AMAT by 0.6 ns (1% of 60 ns). Even at a 100% hit ratio the AMAT cannot fall below the hit time of 3 ns.

7. CPU – Cache – RAM Diagram

The diagram below shows how the CPU, cache and RAM are connected and how a memory request travels between them.

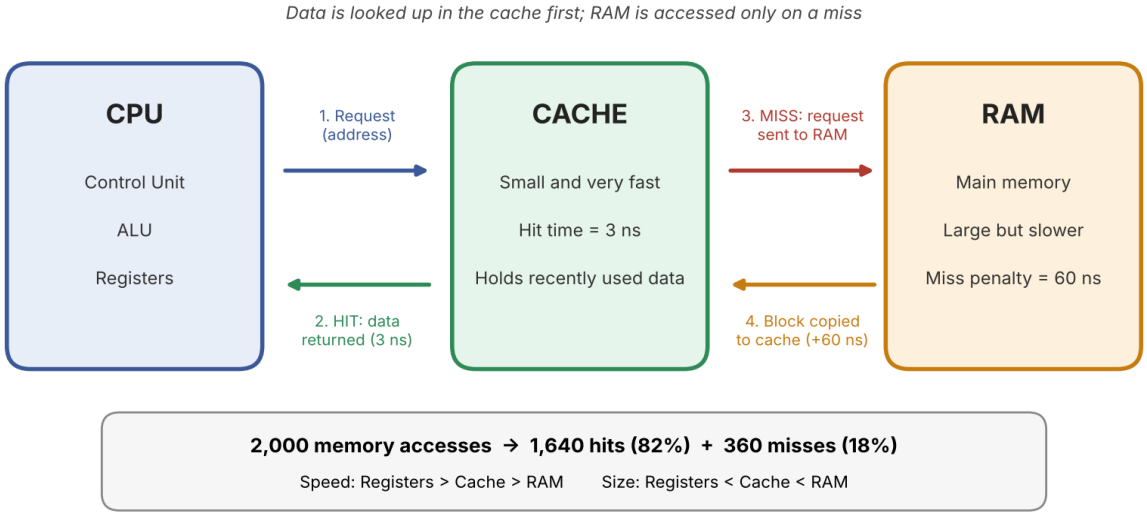


Figure 1: CPU – Cache – RAM memory hierarchy with hit and miss paths

Explanation of data flow

1. **Request:** the CPU needs data and first sends the address to the cache.
2. **Lookup:** the cache checks whether it holds a copy of that data.
3. **Hit:** if the data is present (1,640 of 2,000 accesses), it is returned to the CPU in 3 ns.
4. **Miss:** if the data is absent (360 of 2,000 accesses), the request is forwarded to RAM.
5. **Block transfer:** RAM sends the data, which costs an extra 60 ns. The block is stored in the cache and passed to the CPU, so the next access to it is a hit.

8. Key Formulas Used

Quantity	Formula	Result
Hit Ratio	Hits ÷ Total Accesses	0.82
Miss Ratio	Misses ÷ Total Accesses	0.18
AMAT	Hit Time + (Miss Rate × Miss Penalty)	13.8 ns
AMAT in terms of hit ratio	63 − 60 × Hit Ratio	13.8 ns
Speedup	Time without cache ÷ AMAT	4.35
Reduction in access time	(60 − AMAT) ÷ 60 × 100	77%

9. Observations

The calculated results lead to the following observations:

1. **Most accesses run at cache speed.** With an 82% hit ratio, 1,640 of the 2,000 accesses take only 3 ns. The AMAT is 13.8 ns, far below the 63 ns of a miss, and about 4.35 times better than using RAM alone.
2. **Misses cause most of the delay.** Only 18% of accesses are misses, yet their penalty adds up to 360 × 60 = 21,600 ns out of the total 27,600 ns, which is about 78% of the time. The miss penalty is therefore the main factor that raises the AMAT.
3. **A small gain in hit ratio helps a lot.** Each 1% increase in hit ratio saves 0.6 ns. If the hit ratio rises from 82% to 95%, the AMAT drops from 13.8 ns to 6.0 ns, a reduction of about 57%. Methods that improve the hit ratio, such as a bigger cache or programs with better locality, are therefore very valuable.

10. Conclusion

This case study analysed the performance of a cache for a processor that made 2,000 memory accesses. The hit ratio was found to be 0.82 (82%) and the miss ratio 0.18 (18%). With AMAT = Hit Time + (Miss Rate × Miss Penalty), the average memory access time came to 3 + (0.18 × 60) = 13.8 ns, and this value was confirmed by adding up the time of all the accesses.

The cache made memory access about 4.35 times faster than RAM alone. The study also showed that hit ratio and AMAT are inversely related, and that locality of reference is what allows a small cache to serve most requests. A high hit ratio is therefore the key to a fast memory system.

11. References

1. J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, Morgan Kaufmann.
2. W. Stallings, Computer Organization and Architecture: Designing for Performance, Pearson.
3. M. M. Mano, Computer System Architecture, Pearson.
4. Course lecture notes, Computer Organization and Architecture, Chandigarh University.